

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Пермский национальный исследовательский политехнический
университет»

Электротехнический факультет

Кафедра: Информационные технологии и автоматизированные системы
(ИТАС)

Направление подготовки: 09.03.04 – «Программная инженерия»

ОТЧЕТ

Лабораторная работа №9.

Тема: «Обработка исключительных ситуаций»

По дисциплине «Основы алгоритмизации и программирования»

Вариант 13

Выполнила: студентка группы РИС-25-26

Лаптева Елена Владимировна

Принял: доц. Полякова О.А.

Пермь, 2026

Постановка задачи:

1. Реализовать класс, перегрузить для него операции, указанные в варианте.
2. Определить исключительные ситуации.
3. Предусмотреть генерацию исключительных ситуаций.

Класс-контейнер СПИСОК с ключевыми значениями типа `int`.

Реализовать операции:

- `[]` — доступ по индексу;
- `+` вектор — добавление списка `b` к списку `a` (`a+b`);
- `+` число — добавляет элемент в начало списка.

Варианты реализации: 1, 2

- Вариант реализации 1:

Информация об исключительных ситуациях передается с помощью стандартного типа данных (`int`).

- Вариант реализации 2:

Информация об исключительных ситуациях передается с помощью пользовательского класса (класс `error`).

Анализ задачи:

1. Создать класс СПИСОК (динамический массив целых чисел)
2. Перегрузить 3 операции:
 - `[]` — доступ к элементу по индексу
 - `+` (список) — конкатенация (добавление одного списка к другому)
 - `+` (число) — добавление элемента в начало списка
3. Реализовать обработку исключений двумя способами:
 - Вариант 1: выбрасывать `int` (код ошибки)
 - Вариант 2: выбрасывать объект пользовательского класса `error`
4. Определить исключительные ситуации:
 - Отрицательный индекс
 - Индекс больше размера списка
 - Переполнение (превышение `MAX_SIZE`)
 - И другие ошибки

Код:

Вариант реализации 1

List.h

```
#pragma once
#include<iostream>
using namespace std;

const int MAX_SIZE = 30;

class List {
    int size;
    int* data;

public:
    List();
    List(int s);
    List(const List& other);
    ~List();

    const List& operator=(const List& other);
    int operator[](int i);
    List operator+(const List& other);
    List operator+(int value);

    friend ostream& operator<<(ostream& out, const List& list);
    friend istream& operator>>(istream& in, List& list);
};
```

List.cpp

```
#include "List.h"

List::List() {
    size = 0;
    data = nullptr;
}

List::List(int s) {
```

```

        if (s > MAX_SIZE) throw 1;
        if (s < 0) throw 1;
        size = s;
        data = new int[size];
        for (int i = 0; i < size; i++)
            data[i] = 0;
    }

List::List(const List& other) {
    size = other.size;
    data = new int[size];
    for (int i = 0; i < size; i++)
        data[i] = other.data[i];
}

List::~~List() {
    if (data != nullptr)
        delete[] data;
}

const List& List::operator=(const List& other) {
    if (this == &other) return *this;
    if (data != nullptr) delete[] data;

    size = other.size;
    data = new int[size];
    for (int i = 0; i < size; i++)
        data[i] = other.data[i];
    return *this;
}

int List::operator[](int i) {
    if (i < 0) throw 2;
    if (i >= size) throw 3;
    return data[i];
}

List List::operator+(const List& other) {

```

```

    if (size + other.size > MAX_SIZE) throw 4;

    List result(size + other.size);
    for (int i = 0; i < size; i++)
        result.data[i] = data[i];
    for (int i = 0; i < other.size; i++)
        result.data[size + i] = other.data[i];

    return result;
}

List List::operator+(int value) {
    if (size + 1 > MAX_SIZE) throw 4;

    List result(size + 1);
    result.data[0] = value;
    for (int i = 0; i < size; i++)
        result.data[i + 1] = data[i];

    return result;
}

ostream& operator<<(ostream& out, const List& list) {
    if (list.size == 0)
        out << "Empty list\n";
    else {
        for (int i = 0; i < list.size; i++)
            out << list.data[i] << " ";
        out << endl;
    }
    return out;
}

istream& operator>>(istream& in, List& list) {
    for (int i = 0; i < list.size; i++) {
        cout << "Element [" << i << "]: ";
        in >> list.data[i];
    }
}

```

```
    return in;
}
```

Main.cpp

```
#include "List.h"
#include <iostream>
using namespace std;

int main() {
    setlocale(LC_ALL, "Russian");

    try {
        cout << "=== ТЕСТИРОВАНИЕ КЛАССА LIST ===\n\n";

        // 1. Создание списков
        cout << "1. Создание списка из 3 элементов:\n";
        List list1(3);
        cin >> list1; // Исправлено: теперь оператор >> работает
        cout << "Список 1: " << list1 << endl;

        cout << "\n2. Создание списка из 2 элементов:\n";
        List list2(2);
        cin >> list2; // Исправлено
        cout << "Список 2: " << list2 << endl;

        // 2. Доступ по индексу
        cout << "\n3. Доступ по индексу:\n";
        int index;
        cout << "Введите индекс (0-2): ";
        cin >> index;
        cout << "Элемент с индексом " << index << ": " << list1[index] << endl;

        // 3. Конкатенация списков
        cout << "\n4. Конкатенация списков (list1 + list2):\n";
        List list3 = list1 + list2; // Исправлено: оператор + для списков
        cout << "Результат: " << list3 << endl;

        // 4. Добавление элемента в начало
```

```

    cout << "\n5. Добавление элемента 100 в начало списка:\n";
    List list4 = list1 + 100; // Исправлено: оператор + для числа
    cout << "Результат: " << list4 << endl;

    // 5. Генерация исключений
    cout << "\n6. Генерация исключительных ситуаций:\n";
    cout << "a) Попытка доступа по отрицательному индексу:\n";
    int x = list1[-1]; // Вызовет исключение 2

}

catch (int errorCode) {
    cout << "\n!!! ERROR !!!\n";
    switch (errorCode) {
        case 1: cout << "Ошибка: превышен максимальный размер списка\n"; break;
        case 2: cout << "Ошибка: отрицательный индекс\n"; break;
        case 3: cout << "Ошибка: индекс больше размера списка\n"; break;
        case 4: cout << "Ошибка: переполнение списка\n"; break;
        default: cout << "Неизвестная ошибка\n";
    }
}

return 0;
}

```

Вариант реализации 2

Error.h

```

#pragma once
#include<string>
#include<iostream>
using namespace std;

class error
{
    string str;
public:
    error(string s) { str = s; }
    void what() { cout << str << endl; }
};

```

List.h

```
#pragma once

#include<iostream>

using namespace std;

const int MAX_SIZE = 30;

class List {
    int size;
    int* data;

public:
    List();
    List(int s);
    List(const List& other);
    ~List();

    const List& operator=(const List& other);
    int operator[](int i);
    List operator+(const List& other);
    List operator+(int value);

    friend ostream& operator<<(ostream& out, const List& list);
    friend istream& operator>>(istream& in, List& list);
};
```

List.cpp

```
#include "List.h"
#include "error.h"

List::List() {
    size = 0;
    data = nullptr;
}

List::List(int s) {
    if (s > MAX_SIZE)
        throw error("Error: list exceeds maximum size");
}
```



```

    if (s < 0)
        throw error("Error: negative list size");
    size = s;
    data = new int[size];
    for (int i = 0; i < size; i++)
        data[i] = 0;
}

List::List(const List& other) {
    size = other.size;
    data = new int[size];
    for (int i = 0; i < size; i++)
        data[i] = other.data[i];
}

List::~~List() {
    if (data != nullptr)
        delete[] data;
}

const List& List::operator=(const List& other) {
    if (this == &other) return *this;
    if (data != nullptr) delete[] data;

    size = other.size;
    data = new int[size];
    for (int i = 0; i < size; i++)
        data[i] = other.data[i];
    return *this;
}

int List::operator[](int i) {
    if (i < 0)
        throw error("Error: negative index");
    if (i >= size)
        throw error("Error: index is greater than list size");
    return data[i];
}

```

```

List List::operator+(const List& other) {
    if (size + other.size > MAX_SIZE)
        throw error("Error: list concatenation overflow");

    List result(size + other.size);
    for (int i = 0; i < size; i++)
        result.data[i] = data[i];
    for (int i = 0; i < other.size; i++)
        result.data[size + i] = other.data[i];

    return result;
}

List List::operator+(int value) {
    if (size + 1 > MAX_SIZE)
        throw error("Error: overflow when adding an element");

    List result(size + 1);
    result.data[0] = value;
    for (int i = 0; i < size; i++)
        result.data[i + 1] = data[i];

    return result;
}

ostream& operator<<(ostream& out, const List& list) {
    if (list.size == 0)
        out << "Empty list\n";
    else {
        for (int i = 0; i < list.size; i++)
            out << list.data[i] << " ";
        out << endl;
    }
    return out;
}

istream& operator>>(istream& in, List& list) {

```

```

        for (int i = 0; i < list.size; i++) {
            cout << "Element [" << i << "]: ";
            in >> list.data[i];
        }
        return in;
    }
}

```

Main.cpp

```

#include "List.h"
#include "error.h"
#include <iostream>
using namespace std;

int main() {
    setlocale(LC_ALL, "Russian");

    try {
        cout << "=== ТЕСТИРОВАНИЕ КЛАССА LIST ===\n\n";

        // 1. Создание списков
        cout << "1. Создание списка из 3 элементов:\n";
        List list1(3);
        cin >> list1;
        cout << "Список 1: " << list1 << endl;

        cout << "\n2. Создание списка из 2 элементов:\n";
        List list2(2);
        cin >> list2;
        cout << "Список 2: " << list2 << endl;

        // 2. Доступ по индексу
        cout << "\n3. Доступ по индексу:\n";
        int index;
        cout << "Введите индекс (0-2): ";
        cin >> index;
        cout << "Элемент с индексом " << index << ": " << list1[index] << endl;

        // 3. Конкатенация списков
    }
}

```

```

cout << "\n4. Конкатенация списков (list1 + list2):\n";
List list3 = list1 + list2;
cout << "Результат: " << list3 << endl;

// 4. Добавление элемента в начало
cout << "\n5. Добавление элемента 100 в начало списка:\n";
List list4 = list1 + 100;
cout << "Результат: " << list4 << endl;

// 5. Генерация исключений
cout << "\n6. Генерация исключительных ситуаций:\n";
cout << "a) Попытка доступа по отрицательному индексу:\n";
int x = list1[-1];

}
catch (error e) {
    cout << "\n!!! EXCEPTION CAUGHT !!!\n";
    e.what();
}

return 0;
}

```

Результаты работы:

Вариант реализации 1

```
=== ТЕСТИРОВАНИЕ КЛАССА LIST ===

1. Создание списка из 3 элементов:
Element [0]: 1
Element [1]: 2
Element [2]: 3
Список 1: 1 2 3

2. Создание списка из 2 элементов:
Element [0]: 2
Element [1]: 1
Список 2: 2 1

3. Доступ по индексу:
Введите индекс (0-2): 2
Элемент с индексом 2: 3

4. Конкатенация списков (list1 + list2):
Результат: 1 2 3 2 1

5. Добавление элемента 100 в начало списка:
Результат: 100 1 2 3

6. Генерация исключительных ситуаций:
а) Попытка доступа по отрицательному индексу:

!!! ERROR !!!
Ошибка: отрицательный индекс
```

Вариант реализации 2

```
=== ТЕСТИРОВАНИЕ КЛАССА LIST ===

1. Создание списка из 3 элементов:
Element [0]: 12
Element [1]: 13
Element [2]: 14
Список 1: 12 13 14

2. Создание списка из 2 элементов:
Element [0]: 19
Element [1]: 09
Список 2: 19 9

3. Доступ по индексу:
Введите индекс (0-2): 1
Элемент с индексом 1: 13

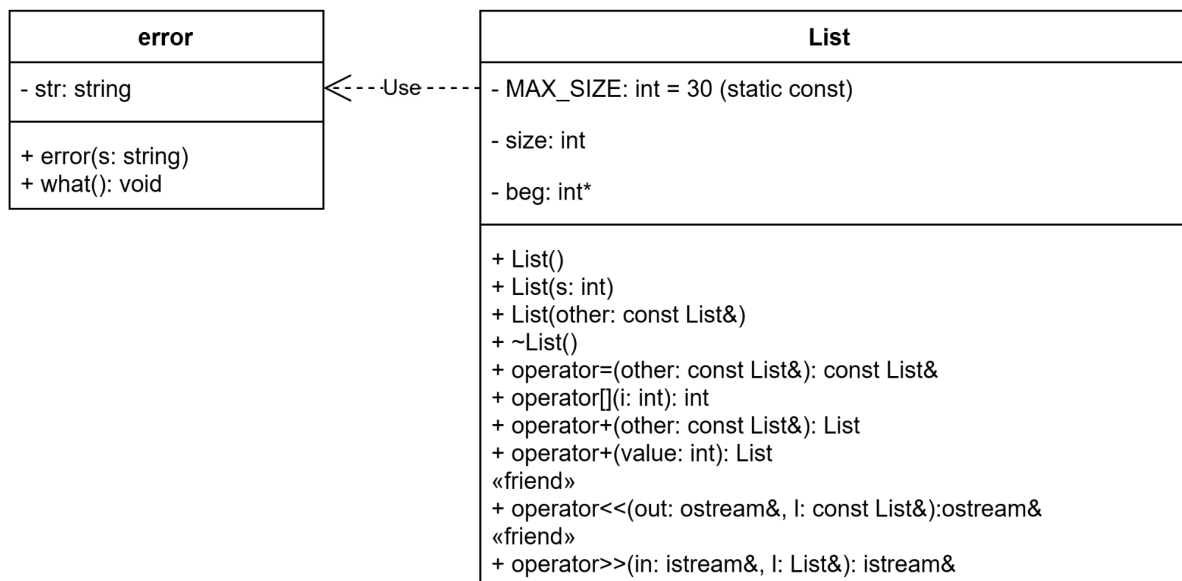
4. Конкатенация списков (list1 + list2):
Результат: 12 13 14 19 9

5. Добавление элемента 100 в начало списка:
Результат: 100 12 13 14

6. Генерация исключительных ситуаций:
а) Попытка доступа по отрицательному индексу:

!!! EXCEPTION CAUGHT !!!
Error: negative index
```

UML:



Вопросы:

1. Исключение в C++ – это непредвиденное или аварийное событие, объект, который система генерирует при возникновении исключительной ситуации.
2. Исключения позволяют разделить вычислительный процесс на обнаружение аварийной ситуации и её обработку. Достоинства: удобно для программ из нескольких модулей; не требуется возвращать значение в вызывающую функцию.
3. Оператор `throw` выражение.
4. Контролируемый блок – это блок `try { }`, в котором выполняется код, где могут возникнуть исключения. Он нужен для того, чтобы связать с ним секцию ловушки `catch`.
5. Секция ловушки `catch` – это обработчик исключения, который перехватывает и обрабатывает исключение, сгенерированное в блоке `try`.
6. Три формы: (тип имя), (тип), (...). Первые две перехватывают конкретные исключения (с доступом к объекту или без), третья – все исключения (должна быть последней).
7. Стандартный класс `exception`.
8. Объявить свой базовый класс-исключение (можно унаследовать от `exception`), а затем создать производные от него классы.
9. Функция `fl()` может генерировать исключения типов `int` и `double`.

10. Функция `f1()` не генерирует никаких исключений.

11. Исключение может генерироваться в любом месте программы, где выполняется оператор `throw` (внутри тублока или в вызываемых функциях).

12.

- Без спецификации: `double area(double a, double b, double c) { ... }`
- Со спецификацией `throw()`: `double area(double a, double b, double c) throw() { ... }`
- Со стандартным исключением (например, `domain_error`): `double area(double a, double b, double c) throw(domain_error) { ... }`
- С собственным классом исключения: `class HeronError {};` `double area(double a, double b, double c) throw(HeronError) { ... }`